Report WebApp MISTRA

Progetto 2024-2025 Applicazioni Dinamiche per il Web

Bazzotti Edoardo: VR518747
Xiao Simone: VR519027

Contenuti

1. Criteri/principi decisi per lo sviluppo dell’applicazione
2. Come si è organizzato DJANGO per la parte CMS
3. Come si è sviluppato il plugin (o estensione) per il quiz
4. Processo di validazione AA WCAG 2.1

Criteri/Principi decisi per lo sviluppo

1. **Headless / API-First**
   a. Tutte le operazioni del quiz esposte tramite API RESTful (GET /random_test_id/, GET /test/{id}/questions/, POST /submit_results/, ecc.) per consentire integrazione con webapp, mobile
   b. Separazione netta fra backend (Django) e frontend (vanilla JS), con payload JSON
2. **UX e Accessibilità**
   a. Interfaccia responsive e semantica: form di avvio, domande presentate una alla volta, feedback immediato.
   b. Conformità ai requisiti WCAG 2.1 di livello AA per garantire l'usabilità a tutti gli utenti.
3. **Modularità e Scalabilità:**
   a. Architettura basata su Django con moduli separati (quiz_plugin) per facilitare lo sviluppo, la manutenzione e future estensioni. L'uso di Django CMS permette una gestione flessibile dei contenuti.
4. **Manutenibilità:**
   a. Codice pulito, commentato e aderente alle best practice di Django, facilitando la comprensione e le modifiche future.

Metodologie per lo sviluppo

Version control con Git

- Gestione efficiente del codice
- Tracciabilità delle modifiche
- Collaborazione continua

Pair programming

- Due sviluppatori, una tastiera
- Migliora la qualità del codice
- Favorisce il trasferimentto di conoscenze

Accessibilità e multi-dispositivo

- Conformità alle linee guida WCAG 2.1
- Esperienza coerente su smartphone, tablet e desktop
- Inclusione per tutti gli utenti

Organizzazione di Django per CMS

🧠 **Django CMS**

📦 Utilizzato per la creazione del CMS
⚙️ Configurato all'interno di un **Virtual Environment**
🚀 Installazione e aggiornamento pacchetti semplificati

venv

🐍 **Virtual Environment**

🔒 Isola i pacchetti del progetto
✅ Garantisce compatibilità tra ambienti
📁 Usato per gestire **Django** e le sue dipendenze

🗄️ **SQLite**

🧾 Database **sqlite3** integrato
💾 Utilizzato per memorizzare dati e contenuti
🧩 Leggero, ideale per ambienti di sviluppo

Organizzazione di Django per CMS

1. **Replicazione dell'attuale Struttura Sito MISTRA** con l'inserimento delle pagine come articoli/documenti all'interno di Django CMS.
2. Utilizzo estensivo dei **plugin Django CMS** per permettere ai medici di aggiungere e modificare documenti/pagine in autonomia.
3. **Ruoli e Permessi**
    a. Creazione di ruoli utente specifici per i medici, con accesso alla parte di amministrazione di Django CMS per la gestione degli articoli (aggiunta, modifica, eliminazione) e dei dati del quiz.
4. **Personalizzazione Amministrazione**: L'interfaccia di amministrazione di Django è stata configurata (admin.py) per ottimizzare la gestione dei modelli del quiz (Test, Domande, Risposte, Esecuzioni), rendendola più intuitiva per i medici.

Sviluppo del Plugin Quiz

Logica backend

Definizione della **struttura** dei dati — **models.py**

Struttura delle pagine amministratore — **admin.py**

Gestione delle **richieste** — **views.py**

Esposizione degli **endpoint** — **urls.py**

Sviluppo del Plugin Quiz

Logica backend

API RESTful: Endpoint per il recupero delle opzioni di sesso, la selezione casuale dei test, il recupero delle domande di un test (con risposte mescolate), l'invio dei risultati del quiz e la generazione del PDF.

1. **Api_sex_options:** Restituisce le opzioni per il sesso in formato JSON.
2. **Api_test_questions**: Dato un ID di test, restituisce le sue domande e risposte in ordine casuale.
3. **Api_submit_results:** Ritorna il risultato finale del test completato.
4. **Download_quiz_pdf**: Genera e restituisce un PDF con i risultati del quiz.
5. **Get_random_test_id**: Seleziona un test casuale dal database e ne restituisce i dettagli base.

Sviluppo del Plugin Quiz

Logica Frontend

1. **Logica Frontend** (quiz.js): Gestisce l'interazione utente, il flusso del quiz domanda per domanda, la raccolta delle risposte, il calcolo della durata e la visualizzazione dinamica dei risultati.
2. **Template** (quiz_plugin.html): Il plugin è integrato nel layout principale di Django CMS tramite un template dedicato, che carica dinamicamente il contenuto del quiz e i relativi script/stili.
3. **Sicurezza**: Incorpora la gestione del token CSRF per le richieste POST, garantendo la sicurezza delle interazioni client-server.

Processo di Validazione AA WCAG 2.1

Strumenti di Validazione Scelti

Test automatici:

1. **Google Lighthouse:** Permette di controllare performance,accessibilità, best practice e SEO. Offre una visione rapida e un punteggio complessivo.
2. **Axe DevTools:** Permette di identificare i problemi di accessibilità che violano AA WCAG 2.1

Test manuali:

1. **Test della tastiera:** Assicurare la navigabilità e l'interazione completa del sito utilizzando esclusivamente la tastiera.
2. **Ingrandimento delle pagine:** Verificare la corretta resa e leggibilità del contenuto a diversi livelli di zoom del browser (es. 200%, 400%).
3. **Controllo del contrasto di colori:** Verificare se il testo era ben leggibile con l'utilizzo di WebAIM Contrast Checker

Processo di Validazione AA WCAG 2.1

Prove effettuate

Le pagine sono state controllate e validate con l'utilizzo di Lighthouse e Axe DevTools, ottenendo:

- **Punteggio Perfetto in Accessibilità con Lighthouse**: Le pagine più importanti e complesse del sito (Home e Quiz) hanno ottenuto un punteggio di >= 90 negli audit di accessibilità,SEO e best practices di Lighthouse, stabilendo una solida base di conformità WCAG 2.1 AA.
- **Zero Problemi Rilevati con Axe DevTools**: Non è stato rilevato alcun problema di accessibilità (0 Issues), inclusi quelli relativi alle "Best Practices" e alle problematiche minori. Dopo il controllo automatico siamo passati a un controllo manuale:
- **Piena Navigabilità da Tastiera**: Abbiamo navigato tutte le pagine del sito utilizzando esclusivamente la tastiera (Tab, Shift+Tab, Invio, Spazio, frecce). Abbiamo verificato la raggiungibilità di tutti i link, bottoni, moduli e la corretta interazione con elementi complessi come il quiz, completandolo fino alla fine.
- **Contrasti Ottimizzati**: È stata migliorata la leggibilità dei testi cruciali grazie all'utilizzo di WebAIM Contrast Checker

Processo di Validazione AA WCAG 2.1

Problemi rilevati e risolti

- **Prove Effettuate:** Lighthouse e Axe DevTools hanno segnalato la mancanza di attributi come title per gli <iframe>.
- **Risultato:** Gli elementi <iframe> (per video e mappe) sono stati resi comprensibili agli screen reader, migliorando la navigazione per gli utenti non vedenti.
- **Prove Effettuate:** Gli audit automatici (Lighthouse, Axe) hanno evidenziato una struttura errata delle intestazioni (es. uso improprio di <h1>, <h6>).
- **Risultato:** La struttura dei titoli è stata riorganizzata per una navigazione più chiara e semantica, a beneficio sia degli screen reader che della scansione visiva del contenuto.

127.0.0.1:8000/it/test/

Offerta Forma... Panopto E-learning Challenges ctf apiSearch Selezione Rapporto Pittini Posizioni Tester

Azienda Ospedaliera Universitaria Integrata di Verona

CENTRO MISTRA

Centro
Multidisciplinare
Infezioni Sessualmente Trasmesse

HOME CHI SIAMO LE INFEZIONI SERVIZI CONTATTI COLLABORA EVENTI LINK UTILI TEST

Mettiti alla prova

DevTools is now available in Italian

Don't show again | Always match Chrome's language | Switch DevTools to Italian

Elements Console Sources Network Performance Memory Lighthouse

13:08:42 - 127.0.0.1:8000

http://127.0.0.1:8000/it/test/

Performance	Accessibility	Best Practices	SEO
99	100	100	91

99

Performance

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

0–49 50–89 90–100